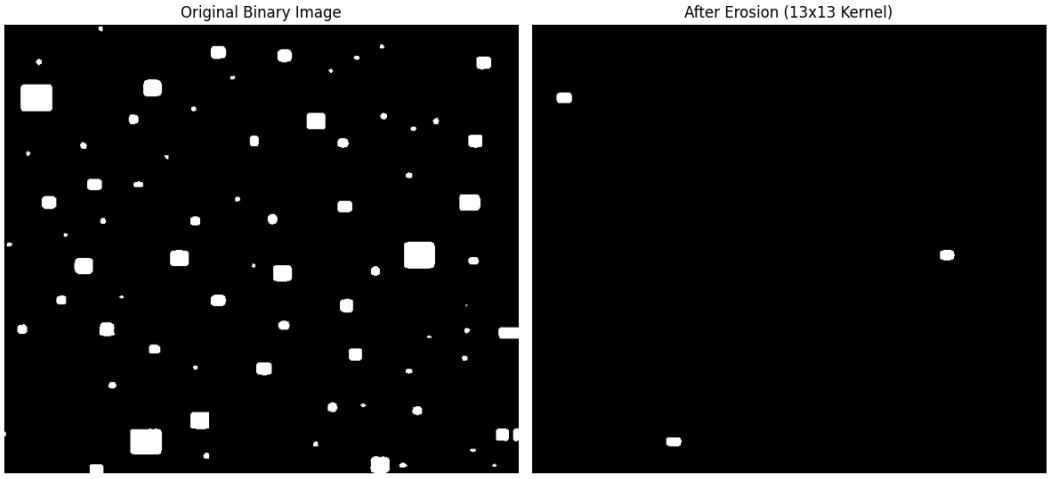
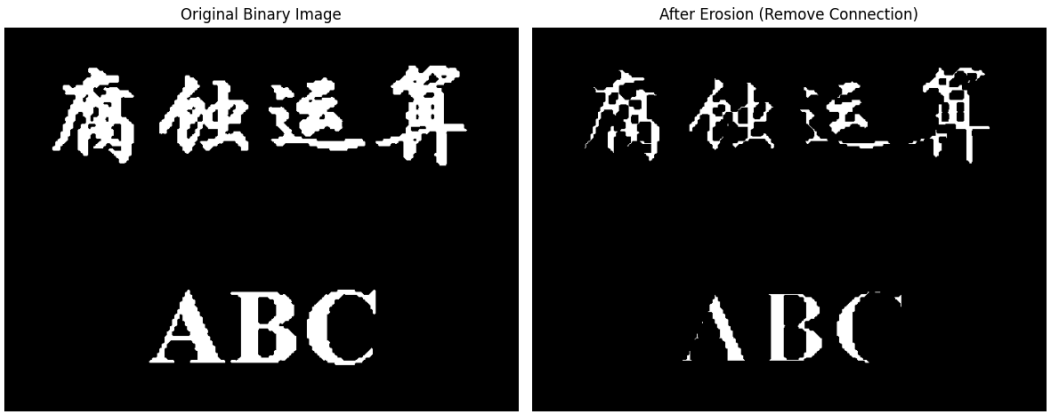
HW18

Problem 1 Erosion

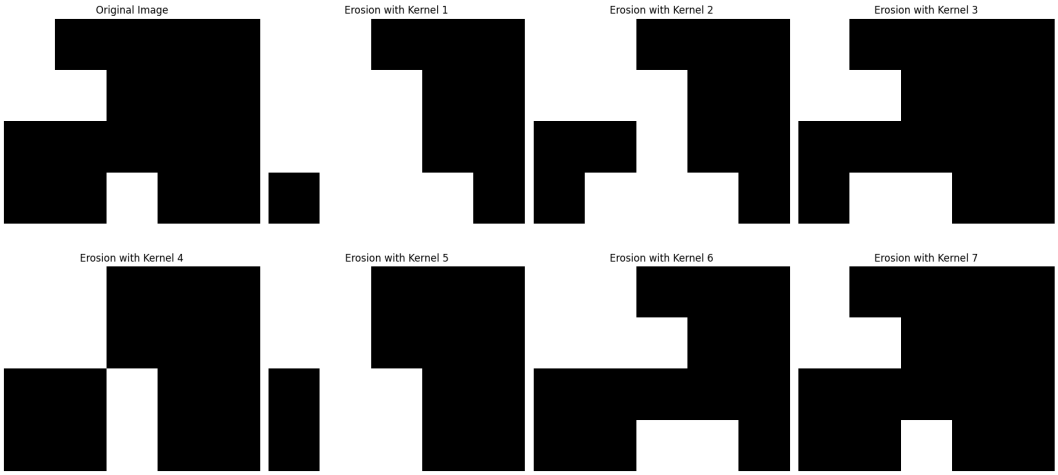
- 1. P20 remove the small square



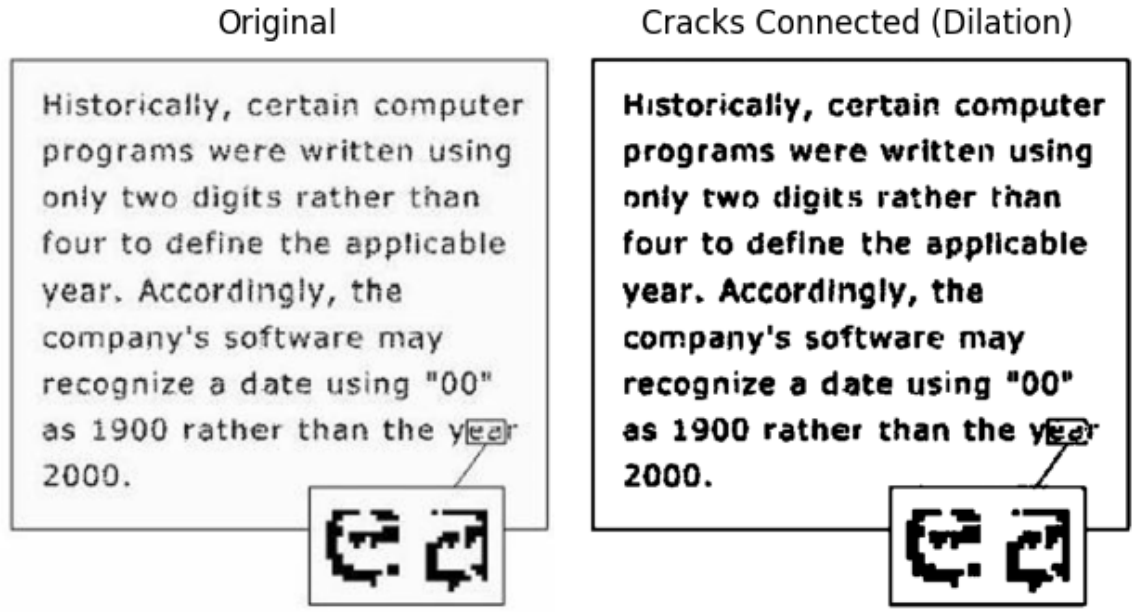
- 2. P21 remove the connection



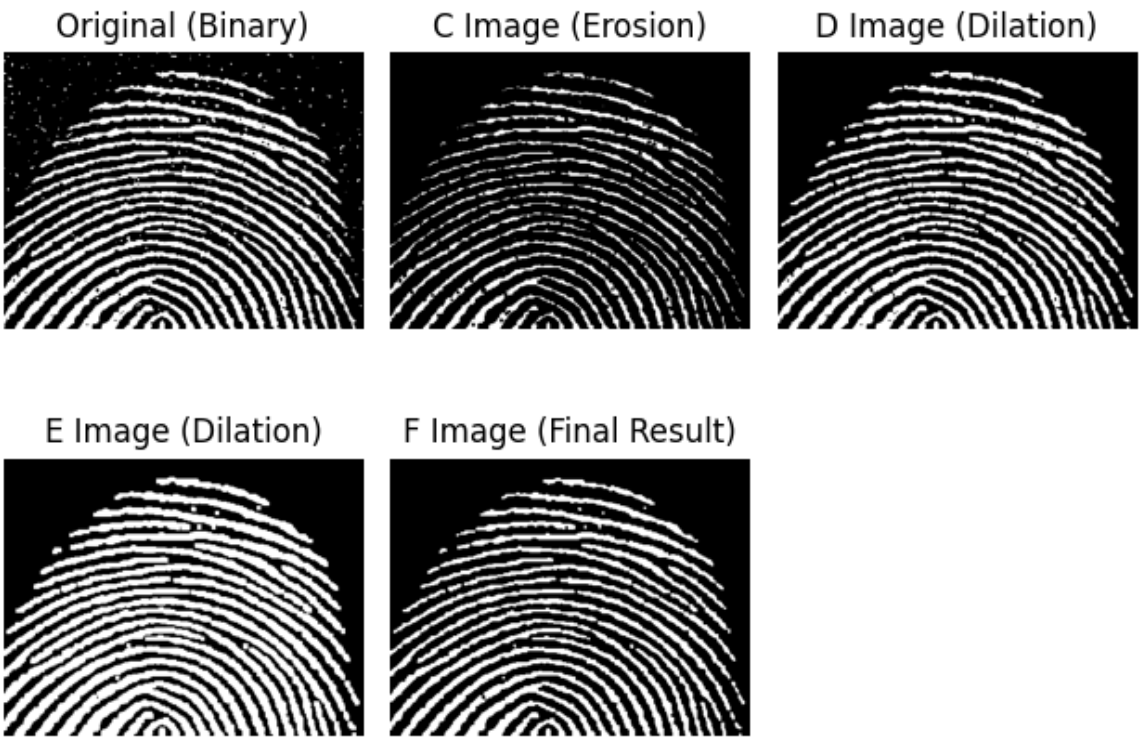
- 3. P22 different elements



Problem 2 Dilation



Problem 3



Code

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ============1=============
# 读取灰度图像
image_path = "1.jpg"
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# 二值化处理
_, binary = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)

# 创建 21x21 的结构元素
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (21, 21))

# 对图像进行腐蚀操作
eroded = cv2.erode(binary, kernel, iterations=1)

# 显示原图和腐蚀后的图像
fig, axs = plt.subplots(1, 2, figsize=(12, 6))
axs[0].imshow(binary, cmap='gray')
axs[0].set_title("Original Binary Image")
axs[0].axis("off")

axs[1].imshow(eroded, cmap='gray')
axs[1].set_title("After Erosion (13x13 Kernel)")
axs[1].axis("off")

plt.tight_layout()
plt.savefig("erosion_task1.png")
plt.close()

# ============2=============
# 读取图像（灰度）
image_path = "2.jpg"
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# 二值化处理
_, binary = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)

# 创建结构元素
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (7, 7))

# 腐蚀操作：去除连接
eroded = cv2.erode(binary, kernel, iterations=1)

# 显示原图与腐蚀后的图像
fig, axs = plt.subplots(1, 2, figsize=(12, 6))
axs[0].imshow(binary, cmap='gray')
axs[0].set_title("Original Binary Image")
axs[0].axis("off")

axs[1].imshow(eroded, cmap='gray')
axs[1].set_title("After Erosion (Remove Connection)")
axs[1].axis("off")
```

```python
plt.tight_layout()
plt.savefig("erosion_task2.png")
plt.close()

# ============3=============
# 原图像（4x5）
image = np.array([
    [0, 1, 1, 1, 1],
    [0, 0, 1, 1, 1],
    [1, 1, 1, 1, 1],
    [1, 1, 0, 1, 1]
], dtype=np.uint8)

# 所有结构元素（2 为锚点/中心）
kernels = [
    np.array([[0, 1, 0],
              [1, 2, 1],
              [0, 1, 0]], dtype=np.uint8),

    np.array([[1, 2, 1],
              [0, 1, 0]], dtype=np.uint8),

    np.array([[2, 1]], dtype=np.uint8),

    np.array([[2],
              [1]], dtype=np.uint8),

    np.array([[2, 1],
              [1, 1]], dtype=np.uint8),

    np.array([[1, 2]], dtype=np.uint8),

    np.array([[2]], dtype=np.uint8)
]

# 提取结构元素的实际掩膜和锚点
def extract_structuring_element(kernel):
    center = tuple(np.argwhere(kernel == 2)[0])        # (row, col)
    binary_kernel = (kernel > 0).astype(np.uint8)      # 将所有非0转为1
    anchor = (center[1], center[0])                    # OpenCV使用(col, row)
    return binary_kernel, anchor

# 图像显示
fig, axs = plt.subplots(2, 4, figsize=(18, 9))
axs = axs.ravel()

# 显示原图（反转颜色）
axs[0].imshow(1 - image, cmap='gray')
axs[0].set_title("Original Image")
axs[0].axis("off")

# 应用每个结构元进行腐蚀（erosion）
for i, kernel in enumerate(kernels):
    se, anchor = extract_structuring_element(kernel)
```

```
    eroded = cv2.erode(image, se, anchor=anchor, iterations=1)

    axs[i+1].imshow(1 - eroded, cmap='gray')  # 黑白反转显示
    axs[i+1].set_title(f"Erosion with Kernel {i+1}")
    axs[i+1].axis("off")

plt.tight_layout()
plt.savefig("erosion_task3.png")
plt.close()

# ============4=============
# 读取图像为灰度图
image = cv2.imread('4.jpg', cv2.IMREAD_GRAYSCALE)

# 反转图像：假设裂缝是黑色，我们反转让前景为黑
inverted = cv2.bitwise_not(image)

# 二值化：前景为1，背景为0
_, binary = cv2.threshold(inverted, 127, 1, cv2.THRESH_BINARY)

# 自定义结构元素（用于连接裂缝）
kernel = np.array([[1, 1, 1],
                   [1, 1, 1],
                   [1, 1, 1]], dtype=np.uint8)

# 膨胀操作连接裂缝
dilated = cv2.dilate(binary, kernel, iterations=1)

# 再反转回来用于显示（黑白语义正确）
final = 1 - dilated

# 显示原图和处理后结果
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(final, cmap='gray')
plt.title("Cracks Connected (Dilation)")
plt.axis("off")

plt.tight_layout()
plt.savefig("dilation_task4.png")
plt.close()

# ============5=============
# 读取指纹图像
image = cv2.imread('5.jpg', cv2.IMREAD_GRAYSCALE)

# 二值化处理：使图像成为黑白图像
_, binary = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)

# 自定义结构元素（选择一个适合的结构元素，例如3x3的矩形）
kernel = np.ones((3, 3), np.uint8)
```

```python
# 1. 腐蚀操作 (c图)
c_image = cv2.erode(binary, kernel, iterations=1)

# 2. 膨胀操作 (d图)
d_image = cv2.dilate(c_image, kernel, iterations=1)

# 3. 再次膨胀操作 (e图)
e_image = cv2.dilate(d_image, kernel, iterations=1)

# 4. 腐蚀操作 (f图) - 最终的图像，消除噪声斑点
f_image = cv2.erode(e_image, kernel, iterations=1)

# 显示各个图像步骤
plt.subplot(2, 3, 1)
plt.imshow(binary, cmap='gray')
plt.title("Original (Binary)")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(c_image, cmap='gray')
plt.title("C Image (Erosion)")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(d_image, cmap='gray')
plt.title("D Image (Dilation)")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(e_image, cmap='gray')
plt.title("E Image (Dilation)")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(f_image, cmap='gray')
plt.title("F Image (Final Result)")
plt.axis("off")

plt.tight_layout()
plt.savefig("fingerprint_task5.png")
plt.close()
```